2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3315-4497-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/COMPSAC69091.2026.00134

# Benchmarking LLM-Based Static Analysis for Secure Smart Contract Development: Reliability, Limitations, and Potential Hybrid Solutions

1st Susan Ștefan-Claudiu
*Department of Computer Science*
*Alexandru Ioan Cuza University of Iași*
Iași, Romania
claudiu_susan@yahoo.com

2nd Arusoaie Andrei
*Department of Computer Science*
*Alexandru Ioan Cuza University of Iași*
Iași, Romania
andrei.arusoaie@uaic.ro

3rd Lucanu Dorel
*Department of Computer Science*
*Alexandru Ioan Cuza University of Iași*
Iași, Romania
dorel.lucanu@uaic.ro

***Abstract*—The irreversible nature of blockchain transactions makes the identification of smart contract vulnerabilities an essential requirement for secure system development. While Large Language Models (LLMs) are increasingly integrated into developer workflows, their reliability as autonomous security auditors remains unproven. We assess whether current generative models are a viable replacement for, or only a complement to, traditional static-analysis tools.**

**Our findings indicate that LLM efficacy is undermined by both inherent lexical bias and a lack of rigorous validation of external data inputs. This reliance on non-semantic heuristics, such as identifier naming, leads to a high frequency of false positives. Furthermore, prompting techniques reveal a trade-off between precision and recall. These results were derived using our custom automated framework, which achieves 92% accuracy in classifying model outputs.**

***Index Terms*—vulnerability detection, static analysis, large language models, smart contracts, solidity, benchmarking**

## I. Introduction

The implementation of smart contracts on blockchain platforms has introduced a "code is law" paradigm, where software vulnerabilities in immutable code lead to financial losses. Traditional static analysis tools such as Slither [5] and Mythril [13] have long served as the first line of defense; however, they are often constrained by rigid rule sets when faced with complex, non-standard code logic. On the other hand, LLMs offer a promising alternative for automated auditing due to their vast training data and ability to generalize to new smart contract implementations.

Integrating LLMs into the blockchain security lifecycle introduces a theoretical paradox. While these models may be able to identify potential exploits, it remains unclear whether they can mimic genuine semantic reasoning or rely solely on probabilistic pattern matching—a phenomenon known as *lexical bias*. Recent findings from Apple [23], show that LLM performance degrades notably when encountering superficial variations in problem descriptions, suggesting an overreliance on lexical cues rather than robust logical derivation.

An extended technical report containing the full experimental tables, the threats-to-validity discussion, the statistical analysis, and the prompt catalogue is available on arXiv [25].

In the context of developing secure smart contracts, a detector that reasons on variable names rather than execution logic is not reliable. This reliance (see, e.g., [24]) makes an LLM a "stochastic auditor": it can be effective when the code matches its training distribution, but it may fail on novel or sanitized logic that lacks descriptive identifiers. To investigate the potential for LLMs to augment existing security issues, this study addresses the following research questions:

**RQ1: Lexical Bias** To what extent do LLMs rely on non-semantic code heuristics, such as descriptive identifier names and comments, rather than semantic execution logic for identifying smart contract vulnerabilities?

**RQ2: Reasoning versus Pattern Matching** Does Chain-of-Thought (CoT) prompting enhance the logical reasoning capabilities of LLMs in vulnerability detection?

**RQ3: Hybrid Integration Performance** How does grounding an LLM with security reports from traditional static analysis tools impact the model's detection range?

**RQ4: Hybrid Integration Bias** Does grounding an LLM with security reports from traditional static analysis tools introduce a measurable bias toward the tool's findings?

**RQ5: Architectural Efficiency** What are the performance and reliability trade-offs when deploying LLMs locally using 4-bit quantization compared to full-precision API-based models for security auditing? Local deployment matters because some auditing clients must keep proprietary contracts on premises and therefore cannot send them to third-party APIs.

We conduct an exploratory empirical investigation on smart contracts security space to answer these research questions. Smart contracts are an ideal case study for vulnerability detection using static analysis techniques because they are *specialized* programs executed in a distributed environment (the blockchain), where code vulnerabilities can result in significant financial losses. Moreover, once deployed, such programs are public and cannot be modified, making the a priori discovery of vulnerabilities highly desirable. Existing vulnerable smart contract benchmarks often contain suggestive names (for variables, functions, contracts) that could hint to

2836-3795/26/$31.00 ©2026 IEEE
DOI 10.1109/COMPSAC69091.2026.00134



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 22:02:18 UTC from IEEE Xplore. Restrictions apply.

LLMs about what they should look for. LLM detections may be inflated by syntactic clues. This motivates our paired-variant protocol.

Answering our research questions requires addressing several challenges: finding *a relevant benchmark for using LLMs in static analysis* – comparable to the existing ones that were designed for traditional static tools; leveraging LLMs to provide *consistent structured output* – even when explicitly requested to adhere to a strict response format, structural inconsistencies can still be encountered, or receive answers with a mismatching terminology; dealing with *ambiguity in evaluation criteria* – what counts as a correct detection can be unclear; tackling *prompt sensitivity and instability* – LLM behavior varies, depending on prompt phrasing; acknowledging the *scale and cost constraints* of LLM evaluation.

*Contributions:* We address these challenges by:

1) An exploratory empirical characterization of how *lexical bias*, *prompting*, and *external integrations* affect LLM-based vulnerability detection in smart contracts.
2) A large-scale evaluation of 15 LLMs, establishing a realistic baseline for AI-enhanced smart contract auditing.
3) A methodological contribution in the form of a paired-variant lexical-bias protocol comparing matched positive/negative variants to isolate superficial lexical cues.
4) Answering **RQ1**–**RQ5** by comparing matched positive/negative variants under controlled prompting and external-tool conditions, and by analyzing the model outputs with our classification pipeline.
5) An engineering contribution in the form of a reusable reproduction infrastructure and an automatic classifier for unstructured LLM outputs, which achieves 92% accuracy and enables reproducible large-scale experiments on LLM-based code analysis.

*Paper Organization:* In Section II, we provide a brief literature review of closely related work. In Section III, we describe our methodology and the infrastructure that we created. In Section IV, we present the evaluation results that we obtained. In Section V, we highlight our key findings. Finally, in Section VI, we present our conclusions and discuss our future plans.

## II. Related Work

Many vulnerable smart-contract collections exist [1], [6]–[8], [10], [11], [16], [17], [20], [22], mostly written in Solidity [15]. We use the one in [1], which follows the taxonomy proposed in [12] (see Table I) and is inspired by the Toyota ITC Benchmark [14]. Its distinctive feature is paired positive/negative variants: a detector should signal the vulnerability only on the positive variant, enabling robust evaluation of whether probabilistic models can distinguish between the two.

Two recent studies are closely related. Chen et al. [4] evaluated GPT and Llama 2 on the Smartbugs curated dataset across 7 vulnerability types, finding satisfactory recall but high false positive rates. Xiao et al. [19] tested five newer models (GPT-4o-mini, Gemini 1.5 Pro [9], Claude 3.5 Sonnet, Yi-large [21], Qwen-plus) on three datasets and similarly reported high false positive rates, partially mitigated by single-vulnerability prompting. Our work differs in scale (15 models), in the use of a paired-variant benchmark that isolates lexical bias, and in the systematic study of how prompting strategy and external tool integration affect detection behavior.

TABLE I
Defect categories

| Category Index | Category Name |
|---|---|
| 1 | Malicious Environment, Transactions or Input |
| 2 | Blockchain/Environment Dependency |
| 3 | Exception & Error Handling Disorders |
| 4 | Denial of Service |
| 5 | Resource Consumption & Gas Issues |
| 6 | Authentication & Access Control Vulnerabilities |
| 7 | Arithmetic Bugs |
| 8 | Bad Coding and Language Specifics |
| 9 | Environment Configuration Issues |
| 10 | Eliminated/Deprecated Vulnerabilities |

## III. Methodology

Our research methodology is structured into four stages: *the preparation of suitable code examples*; *designing the prompts*; *the analysis of the files using our various AI models*; and *the classification of detections in various defect categories*. The first two stages were performed manually. The large number of models and vulnerability types led us to implement a reusable framework for automating the last two stages.

The methodology flowchart is provided in the extended technical report [25]. The study focuses on Ethereum smart contracts, so we chose Solidity, the language that dominates the deployed smart-contract market and is the only language with paired positive-negative data sets. This choice also reflects the blockchain-specific and language-specific nature of our study and of the benchmark, which are tailored to the execution model, semantics, and tooling ecosystem of smart contracts. In addition, because of its popularity, Solidity is likely to be more frequently represented in LLM training data sets than other smart-contract languages such as Vyper. However, the methodology transfers in principle to other smart-contract languages. The closest extension is Vyper since it is EVM-compatible and has similar ASTs. Rust-based chains would require a new paired benchmark, and Move would invalidate several defect categories under its resource semantics. Producing such benchmarks is left to future work.

*Stage 1, preprocessing the collection of defect examples:* We use the test suite from [1] due to its wide variety of defect categories and its positive/negative pairs for each vulnerability.

This comprehensive range of defect categories allows us to test the capabilities of LLMs in various scenarios. The negative variants enable us to test whether models return false alarms.

In addition, the tests in the selected suite are annotated. The defect contained in each code file (for positive variants) or absent from each file (for negative variants) is clearly indicated in the file name or the name of the vulnerable function. This feature facilitates metric computation. However, it may also bias AI models toward a specific defect type, since annotated defect category names can act as cues. Such detections may



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 22:02:18 UTC from IEEE Xplore. Restrictions apply.

not be fair, because these annotations are not present in practice. To avoid this, but also to measure lexical bias, we performed some additional preprocessing of the dataset, such as renamings (contracts, methods, parameters, or variables).

The original suite contains **208** code examples distributed across **10** generic defect categories and **52** specific vulnerability types. To accommodate our automated classification pipeline, we excluded all examples from **3** specific categories (see the extended technical report [25] for details). This filtering resulted in the final dataset for our study, comprising **176** code examples across **49** specific defect categories.

*Stage 2, designing the prompts:* We designed two prompts. The first requests an analysis report from the model without any additional instructions besides asking it to assume a "senior Smart Contract Security Auditor specializing in Solidity" role. For the second, we present the LLM with a structured approach with additional instructions that ask for additional internal validation of the findings. In prompt engineering (see, e.g, [3], [18]), these two prompts correspond to *Zero-Shot* and *CoT* prompting. We focus on these two canonical prompting families and exclude few-shot, since per-vulnerability exemplars would leak defect-type cues. For each prompt, we also generated a Slither-augmented variant supplying the model with Slither's security report; in the CoT case, we additionally instruct the model to attempt to disprove Slither's detections to mitigate bias. These prompts drive the experiments in Section IV. The full prompts — including the structured-output system prompt used for models without schema-enforcement parameters — are reproduced in [25].

*Stage 3, analyze defect examples:* We used official client libraries to communicate with both remote and local model endpoints and prompted the models to return structured detections: defect type, general defect definition, and how the defect manifests in the contract being analyzed. The resulting analysis report is then stored in a database. To handle transient API errors, such as server overload or rate limiting, failed analysis runs were stored and subsequently retried. This ensures complete data collection and prevents temporary issues from being incorrectly marked as undetected vulnerabilities.

*Stage 4, classify the results of the analysis:* This stage begins by automatically assigning specific defect types to the detections generated during the processing stage. Due to the large number of detections that we obtained, manually classifying each detection is not feasible. We used an automatic classification methodology based on semantic similarity [2], which in its current iteration achieves 92% accuracy; a detailed description is provided in [25] Then, relevant performance metrics are computed based on the detections and their assigned labels. We also measure the number of instances where an LLM failed to produce structured output and the detections that match the ones provided by Slither.

## IV. Evaluation Process

Given the description of *Stage 2* in our methodology, we evaluate how each combination performs on our dataset: Zero-Shot with and without Slither input, and CoT with and without Slither input. As described in *Stage 1*, we preprocessed the dataset in [1] to eliminate the most basic forms of lexical bias. To differentiate between the original, unprocessed dataset from [1] and the processed one used here, we refer to them as Dataset 1 and Dataset 2, respectively. We performed the experiments with their objectives shown in Table II following the process described in [25].

TABLE II
Experimental Design and Objectives

| Exp. | Dataset | Prompting | Slither | Primary Objective |
|---|---|---|---|---|
| 1 | Dataset 1 | Zero-Shot | No | Establish baseline |
| 2 | Dataset 2 | Zero-Shot | No | Quantify Lexical Bias |
| 3 | Dataset 2 | CoT | No | Evaluate reasoning benefit |
| 4 | Dataset 2 | Zero-Shot | Yes | Hybrid tool integration |
| 5 | Dataset 2 | CoT | Yes | Measure Slither Bias |

### A. Model Selection

The primary criterion for model selection was the feasibility of long-term integration into custom static analysis workflows. To keep reasonable financial costs, we fixed a strict pricing ceiling of $5 per 1 million input/output tokens. This threshold implied the exclusion of flagship models from numerous providers, such as Claude 4.5 Opus (Anthropic), which we categorize as too costly for large-scale auditing tasks. After this filtering, we prioritized paired-series models, thus enabling direct size and reasoning comparisons, and excluded those without a counterpart to keep the matrix consistent.

The final list of included models is shown in Table III, and the API providers used to access them are listed in [25].

### B. Metrics

We adopt the performance metrics defined in [1]. The *Recall* measures the proportion of actual positive instances that were correctly identified by the model. The *False Positive Rate (FPR)* represents the proportion of negative instances that are incorrectly classified as positive. Each detection is labeled as a true positive (TP) if it correctly identifies a vulnerability in a positive variant, a true negative (TN) if it correctly reports no vulnerability in a negative variant, a false positive (FP) if it reports a vulnerability in a negative variant, or a false negative (FN) if it misses a vulnerability in a positive variant. The *Balanced Accuracy* tells us how well a tool finds positives and avoids false alarms. A higher value for $BA$ is better. Here are the definitions we use:

$$FPR = FP/(FP + TN), \quad Recall = TP/(TP + FN),$$
$$BA = \big(Recall + (1 - FPR)\big)/2.$$

TABLE III
Models and their developers included in our study

| Developers | Models |
|---|---|
| DeepSeek | DeepSeek Chat, DeepSeek Reasoner, DeepSeek R1 14b |
| Mistral | Magistral Medium, Magistral 24b (local and API), Devstral, Devstral Small (local and API) |
| xAI | Grok 4.1 fast reasoning, Grok 4.1 fast non-reasoning |
| OpenAI | GPT Oss 120b, GPT Oss 20b (local and API) |
| Alibaba Cloud | Qwen3 30b (thinking), Qwen3 30b (instruct), Qwen3 235b (thinking), Qwen3 235b (instruct) |



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 22:02:18 UTC from IEEE Xplore. Restrictions apply.

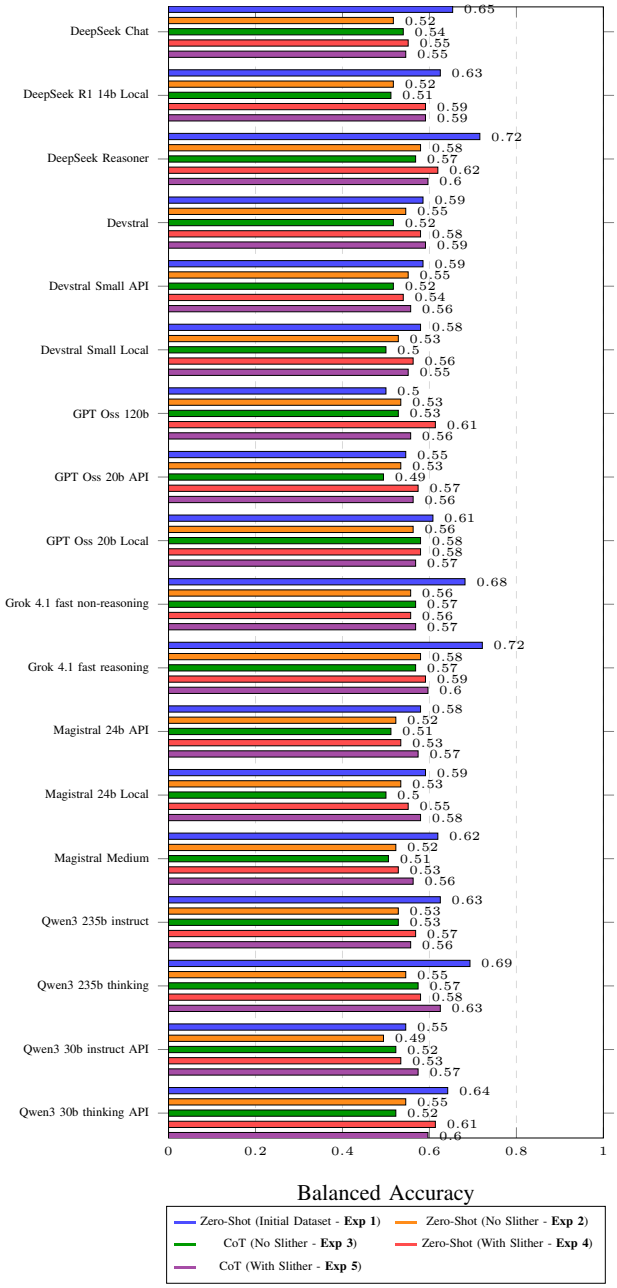



Fig. 1. Balanced accuracy across all experiments.

### C. Analysis of the Results

Our analysis of the results addresses each research question, with aggregated changes in balanced accuracy reported. Full experiment-level and per-category results are available in the extended technical report [25].

*1) Quantifying the lexical bias between the two datasets:* To answer **RQ1**, we compare Experiment 1 (which uses the original defect examples containing syntactical clues that LLMs might exploit) against all other experiments (which use the preprocessed defects suite with syntactical clues removed).

TABLE IV
COMPARISON OF BA BETWEEN EXPERIMENTS 2–5 AND EXPERIMENT 1. VALUES INDICATE LOWER DETECTION PERFORMANCE IN EXPERIMENTS 2–5 COMPARED TO EXPERIMENT 1, SUGGESTING THAT IN EXPERIMENT 1 THE SUPERIOR ACCURACY IS DUE TO LEXICAL BIAS.

| Exp $i$ vs. Exp $j$ | Result | $\Delta$BA | Status |
|---|---|---|---|
| Exp 2 vs. Exp 1 | WORSE | -0.077652 | |
| Exp 3 vs. Exp 1 | WORSE | -0.085543 | |
| Exp 4 vs. Exp 1 | WORSE | -0.046086 | |
| Exp 5 vs. Exp 1 | WORSE | -0.041351 | |

TABLE V
COMPARISON OF BA BETWEEN EXPERIMENTS WITH ZERO-SHOT AND COT. VALUES DO NOT INDICATE AN IMPROVEMENT WHEN USING COT.

| Exp $i$ vs. Exp $j$ | Result | $\Delta$BA | Status |
|---|---|---|---|
| Exp 3 vs. Exp 2 | WORSE | -0.007891 | |
| Exp 5 vs. Exp 4 | BETTER | 0.004735 | |

The raw data shows that Experiment 1 consistently achieves higher BA than Experiments 2–5 (cf. Table IV), showing the presence of *lexical bias*. This can be seen in Figure 1, where the Zero-Shot blue bars have better scores. This suggests that specific code wording may enable LLMs to detect defects through lexical cues rather than by semantic reasoning.

*2) Analyzing the impact of using CoT prompting:* To answer **RQ2**, we compare Experiments 2 and 3 (Zero-Shot vs. CoT without Slither) and Experiments 4 and 5 (Zero-Shot vs. CoT with Slither) to assess whether CoT outperforms Zero-Shot. Table V shows that without Slither, CoT performs slightly worse, while with Slither, CoT performs better.

However, on aggregate, the $\Delta$BA–defined as the numerical difference in BA between two experimental configurations–CoT and Zero-Shot –is within per-model variance ($\pm 0.06$).

Notably, the effect of CoT varies across models: for Qwen3 30b instruct API, Qwen3 235b thinking, and Magistral 24b API, CoT improves BA by approximately 0.04, while for other models (e.g., GPT Oss 120b) it decreases BA by up to 0.06.

In addition to the above, CoT increased the computational overhead, as models were required to generate an internal chain of reasoning. The time that was necessary for the models to respond increased by up to **10** seconds. Almost all models queried via API did not suffer shifts in structured output consistency compared to Experiment 2. However, a general increase in invalid outputs was noticed for the local models, the number of such instances increasing by between **10** and **20**, effectively doubling for most models.

*3) Assessing the performance induced by integrating LLMs with Slither:* An answer for **RQ3** requires us to evaluate the performance boost introduced when LLMs are supplemented with external security reports, such as those from Slither. Therefore, we compare Zero-Shot with/without Slither (Exp 2 vs. Exp 4) and CoT with/without Slither (Exp 3 vs. Exp 5).

The results are shown in Table VI. The results indicate that incorporating Slither input significantly improves accuracy.

*4) Assessing the bias induced by grounding LLMs with Slither detections:* To answer **RQ4** we compare the matched detections between LLMs and Slither, that is, we count the number of LLM detections that match Slither detections in all



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 22:02:18 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
COMPARISON OF BA BETWEEN EXPERIMENTS WITH/WITHOUT SLITHER INPUT. VALUES FOR COT EXPERIMENTS 4 AND 5 DO INDICATE AN IMPROVEMENT WHEN USING SLITHER INPUT FOR LLMS.

| Exp $i$ vs. Exp $j$ | Result | $\Delta$BA | Status |
|---|---|---|---|
| Exp 4 vs. Exp 2 | BETTER | 0.031566 | |
| Exp 5 vs. Exp 3 | BETTER | 0.044192 | |

scenarios. This analysis differs from the preceding evaluations, so we report the full results in Figure 2. Note that, the number of matching detections is higher when Slither is used.

Without an explicit internal validation step, models in Experiment 4 often adopted Slither's findings with limited additional scrutiny, leading to substantial increases in matching detections. This reliance on external data was most evident in models like Qwen3 30b thinking API, which saw a **354%** increase in detections matching Slither's report compared to its standalone baseline. This suggests that in Zero-Shot mode, the model functions less as an autonomous auditor and more as a parser for the provided deterministic tool output.

In the CoT experiments, models were instructed to follow a structured approach for validating or challenging Slither's findings through an *adversarial pressure test*. This test required models to determine whether a Slither finding was benign or to identify exploits in functions Slither marked as safe. However, the results indicate that CoT often functioned as a bias amplifier rather than a skeptical filter. Instead of using the reasoning chain to identify false leads, many models used the additional computational steps to generate logical-sounding justifications for the tool's detections to maintain token-to-token coherence. DeepSeek R1 14b Local demonstrated this clearly, with Slither-aligned detections increasing by **276%** in the CoT hybrid mode. This failure to maintain an independent adversarial stance suggests that the deterministic signal from a known security tool carries a higher weight in the model's attention mechanism than hypothetical adversarial instructions.

In Zero-Shot, the bias appears consistent with the model's attention mechanism prioritizing structured, deterministic input (Slither reports) over ambiguous raw code. In CoT, the bias appears to stem from instructional alignment; because the model is tasked with analyzing the report, it may treat agreement with the tool as a signal of task completion. This "Stochastic Mirror" effect raises a significant concern for agentic AI systems. If an autonomous agent cannot effectively question its toolset or identify semantic vulnerabilities that lie outside rigid rule-sets, it cannot provide the adversarial redundancy required for an effective audit process.

*5) The feasibility of running models locally:* Because proprietary contracts cannot be sent to third-party APIs under that on-premises constraint, we compare local 4-bit deployment against full-precision API-based models for security auditing. To answer **RQ5** we evaluate the architectural and operational trade-offs of deploying LLMs locally using 4-bit quantization vs. using full-precision API-based models. Our findings reveal that local deployment is feasible for auditing tasks, but it introduces a class of persistent errors that differentiate them

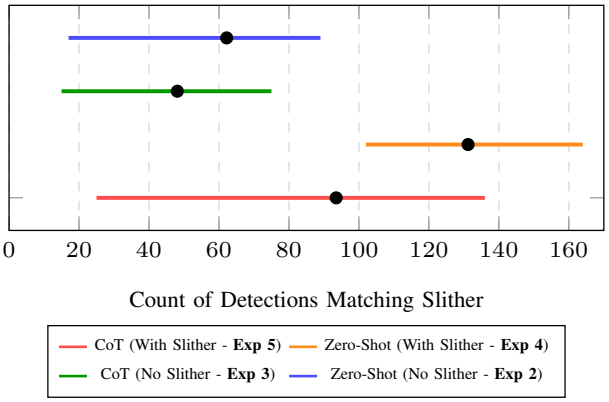



Fig. 2. Slither bias summary. Each horizontal line spans the minimum to maximum matched detections within an experiment, and the black marker indicates the mean number of LLM detections matching Slither for each experiment.

from the remote services. While API-based models occasionally suffered from issues like server overloads, which can be resolved via automated retries, local models Devstral Small and Magistral 24b exhibited deterministic failures on a subset of contracts that persisted across re-runs. Also, local runs timed out due to hardware, exceeding the 5 minutes limit.

Comparing local and API-based models reveals the impact of quantization on performance. API models consistently outperform their local counterparts in recall — for instance, Devstral Small reached 55.68% via API versus 48.86% locally in Experiment 1. Local models also produce more noise: Magistral 24b yielded a 34.09% FPR locally compared to 25.00% via API in the same experiment. Finally, local processing is slower, with Magistral 24b averaging 44 624.74 ms per contract in Experiment 3 against just 2 949.55 ms for APIs.

### D. Addressing the Automatic Classifier Error Rate

To assess the impact of the corresponding **8%** error rate of our automatic classifier, we identified its likely failure modes.

Our classifier, which depends on semantic similarity, often fails to recognize true positives that are phrased in atypical ways, resulting in their misclassification as false negatives. This primary error mode would artificially lower the reported Recall, Precision, and Accuracy metrics. A secondary, less likely error involves the classifier failing to match a FP from an LLM to any known vulnerability, causing it to be misclassified as a TN. This would lead to an underestimation of the FPR.

A per-research-question breakdown of the impact of the classifier error rate is provided in the technical report [25].

## V. KEY TAKEAWAYS

### A. Enhanced Grounding for Hybrid Solutions

Directly grounding models with security reporting (e.g., Slither) induces a significant grounding bias, providing little benefit over running analysis tools and LLMs independently. Because models cannot currently function as reliable standalone detection tools, external integrations are mandatory; however, providing high-level findings often leads the model to act as a "stochastic mirror" rather than a critical auditor.



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 22:02:18 UTC from IEEE Xplore. Restrictions apply.

We propose shifting the grounding layer towards deterministic contract representations, such as the Control Flow Graph (CFG) and Static Single Assignment (SSA) forms. While Slither computes this information internally, the current Slither MCP implementation does not expose it through its endpoints.

### B. A multi-staged prompting approach

In addition to the required improvements in the grounding mechanism, our results demonstrate that, for agentic security workflows, a single prompting strategy may not suffice. Instead, the most effective systems should adopt a tiered architecture: use Zero-Shot prompting for broad discovery to capture a wide range of potential vulnerabilities, followed by a CoT verification pass to refine the results. This allows the system to maintain a high discovery range (Recall) while using CoT to reduce false positives. By using both approaches and dividing them into different stages, agentic solutions can achieve a more reliable and human-readable audit trail without the performance drop typically seen in single-pass reasoning.

## VI. Conclusion and Future Works

We presented a large-scale evaluation of 15 LLMs and an automated classification pipeline applied to paired positive/negative Solidity examples. The experiments demonstrate four practical conclusions: LLM detections are sensitive to lexical cues; Chain-of-Thought prompting reduces false positives at the cost of recall and latency; grounding with static-analysis reports increases agreement but can introduce grounding bias; and 4-bit local deployments currently suffer quantization-induced reliability issues.

Taken together, these results indicate that LLMs are not yet reliable as standalone security auditors but can provide value within carefully designed hybrid pipelines. Promising directions include grounding models on deterministic program representations (e.g., CFG/SSA) rather than high-level report text; adopting multi-stage pipelines that separate broad discovery from verification; and improving local-model robustness to mitigate quantization loss. Planned future work includes: • extending the evaluation to real-world contracts (including deployed samples from ecosystems such as Solana), • exposing finer-grained representations from Slither (via an MCP extension), • exploring property-based test generation for formal verifiers, • extending the study to other smart-contract languages such as Rust, and • investigating methods to reduce logic loss in compressed, on-premise models.

## Acknowledgments

This work was supported by the *CHIST-ERA-23-SMARTC-01-SCEAL* project, by a grant from the Romanian Ministry of Research, Innovation and Digitization, CNCS/CCCDI - UEFISCDI, project no. 86/2025 ERANET-CHISTERA-IV-SCEAL, within PNCDI IV.

## References

[1] Arusoaie, A., Susan, S.: Towards trusted smart contracts: A comprehensive test suite for vulnerability detection. Empir. Softw. Eng. **29**(5), 117 (2024). https://doi.org/10.1007/S10664-024-10509-W

[2] Chandrasekaran, D., Mago, V.: Evolution of semantic similarity—a survey. ACM Computing Surveys (Csur) **54**(2), 1–37 (2021)

[3] Chen, B., Zhang, Z., Langrené, N., Zhu, S.: Unleashing the potential of prompt engineering in large language models: a comprehensive review (2024), https://arxiv.org/abs/2310.14735

[4] Chen, C., Su, J., Chen, J., Wang, Y., Bi, T., Yu, J., Wang, Y., Lin, X., Chen, T., Zheng, Z.: When ChatGPT meets smart contract vulnerability detection: How far are we? ACM Transactions on Software Engineering and Methodology (2023)

[5] Feist, J., Greico, G., Groce, A.: Slither: A static analysis framework for smart contracts. In: Proceedings of WETSEB '19, pp. 8–15. IEEE Press, Montreal, Quebec, Canada (2019). https://doi.org/10.1109/WETSEB.2019.00008

[6] Ferreira, J., Durieux, T., Maranhao, R.: Smartbugs wild. https://github.com/smartbugs/smartbugs-wild (2020)

[7] Ferreira, J., Salzer, G.: Smartbugs curated. https://github.com/smartbugs/smartbugs-curated (2023)

[8] Ferreira, J.a.F., Cruz, P., Durieux, T., Abreu, R.: Smartbugs: A framework to analyze Solidity smart contracts. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering. p. 1349–1352. ASE '20, ACM, New York, NY, USA (2020). https://doi.org/10.1145/3324884.3415298

[9] Gemini Team: Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context (2024), https://arxiv.org/abs/2403.05530

[10] Ghaleb, A., Pattabiraman, K.: Solidifi benchmark. https://github.com/DependableSystemsLab/SolidiFI-benchmark (2020)

[11] NCC-Group: Decentralized application security project. https://dasp.co/ (2018), accessed: 2023-04-04

[12] Rameder, H., di Angelo, M., Salzer, G.: Review of automated vulnerability analysis of smart contracts on Ethereum. Frontiers in Blockchain **5** (2022). https://doi.org/10.3389/fbloc.2022.814977

[13] Sharma, N., Sharma, S.: A survey of mythril, a smart contract security analysis tool for evm bytecode. International Journal of Advanced Research in Computer Science **13**, 51003–51010 (2022)

[14] Shiraishi, S., Mohan, V., Marimuthu, H.: Test suites for benchmarks of static analysis tools. In ISSREW 2015. pp. 12–15 (2015). https://doi.org/10.1109/ISSREW.2015.7392027

[15] Solidity documentation. https://docs.soliditylang.org/en/v0.8.16/, accessed: 2022-09-01

[16] SunWeb3Sec: Defihacks. https://github.com/SunWeb3Sec/DeFiHackLabs/ (2023)

[17] Smart contract weakness classification and test cases. https://swcregistry.io/, accessed: 2023-04-04

[18] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D.C.: A prompt pattern catalog to enhance prompt engineering with ChatGPT. arXiv preprint arXiv:2302.11382 (2023)

[19] Xiao, Z., Wang, Q., Pearce, H., Chen, S.: Logic meets magic: Llms cracking smart contract vulnerabilities. arXiv preprint arXiv:2501.07058 (2025)

[20] Xu, Z., Ren, M.: Smart-contract-benchmark-suites: A unified dataset. https://github.com/renardbebe/Smart-Contract-Benchmark-Suites (2021)

[21] Young, A., et al.: Yi: Open foundation models by 01.AI (2025), https://arxiv.org/abs/2403.04652

[22] Zhang, Z., Zhang, B., Xu, W., Lin, Z.: Web3bugs. https://github.com/ZhangZhuoSJTU/Web3Bugs/tree/main (2022)

[23] P. Shojaee, I. Mirzadeh, K. Alizadeh, M. Horton, S. Bengio, and M. Farajtabar, "The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity," in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2025.

[24] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proc. 2021 ACM Conf. Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623.

[25] S. Susan, A. Arusoaie, and D. Lucanu, "Benchmarking LLM-Based Static Analysis for Secure Smart Contract Development: Reliability, Limitations, and Potential Hybrid Solutions — Extended Technical Report," arXiv:2605.11163, 2026. https://arxiv.org/abs/2605.11163



Authorized licensed use limited to: Nazarbayev University. Downloaded on November 03,2026 at 22:02:18 UTC from IEEE Xplore. Restrictions apply.